AJ_24038155_dialect_v1.0_to_v1.1 update

Change specification from AJ_24038155 Dialect v1.0 to v1.1

Status

This document records the v1.0 to v1.1 update to the frozen AJ_24038155 Dialect v1.0 specification. All v1.0 lexical, syntactic and semantic rules remain normative unless explicitly changed below.

Version v1.1 is introduced under the v1.0 versioning rule because it adds a literal form and a general member-method invocation form and changes the canonical spelling of dictionary retrieval.

1. Native null literal

1.1 Canonical syntax

```
NONE
```

Keywords remain case-insensitive during parsing, therefore `NONE`, `None` and `none` denote the same literal. Canonical output uses uppercase `NONE`.

1.2 Grammar

```
none_literal ::= "NONE" ;
```

The v1.1 literal rule is therefore:

```
literal ::=
integer_literal
| float_literal
| typed_integer
| typed_float
| string_literal
| char_literal
| boolean_literal
| none_literal
| tuple_literal
| typed_list_literal
| mixed_list_literal
```

```
| set_literal
| empty_literal ;
```

1.3 Python translation

```
NONE
```

translates deterministically to:

```
None
```

`NONE` is a value. It is not an identifier and is not a function call.

1.4 Backwards compatibility

The historical compatibility form:

```
CALL NULL()
```

remains accepted by the implementation for existing v1.0 material. New canonical v1.1 pseudocode should use `NONE`.

2. Property access and member-method invocation

v1.1 explicitly distinguishes property access from invocation by the presence of parentheses.

2.1 Property access

```
object.property
```

Examples:

```
transaction.quantity
node.left
drone.destination
```

Grammar:

```
property_access ::=
primary_expression , "." , identifier ;
```

Property access reads a member value and performs no invocation.

2.2 Member-method invocation

```
object.method(arguments)
```

Examples:

```
stack.pop()
drone.move(nextMove)
registry.GET(name)
```

Grammar:

```
member_call_expression ::=
primary_expression ,
"." ,
identifier ,
"(" ,
[ argument_list ] ,
")" ;
```

The method identifier remains case-sensitive. Parentheses distinguish a method invocation from property access:

```
x.item       COMMENT property access
x.item()     COMMENT zero-argument method invocation
```

A member call may be used as an expression or as a standalone statement.

3. Dictionary retrieval

3.1 Canonical v1.1 syntax

Dictionary lookup now uses the general member-call notation:

```
dictionary.GET(key)
dictionary.GET(key, default_value)
```

Examples:

```
value <- visited.GET(node)
count <- frequency.GET(character, 0)
```

3.2 Semantic resolution

`GET` is not globally interpreted as dictionary retrieval by spelling alone.

The source form:

```
receiver.GET(...)
```

is first parsed as an ordinary member-method invocation. It resolves to the dictionary intrinsic only when static evidence establishes that `receiver` is dictionary-backed, for example:

```
frequency <- EMPTY DICTIONARY
count <- frequency.GET(character, 0)
```

or:

```
FUNCTION lookup ACCEPTS(values AS DICTIONARY, key)
RETURN values.GET(key)
ENDFUNCTION
```

or a class property declared `AS DICTIONARY`.

If the receiver is not dictionary-backed, `GET` remains an ordinary user-defined method name. Thus this is valid and distinct:

```
CLASS Registry
METHOD GET ACCEPTS(name)
...
ENDMETHOD
ENDCLASS
```

3.3 Dictionary GET argument rule

When resolved as the dictionary intrinsic, `GET` accepts exactly one or two arguments:

```
dictionary.GET(key)
dictionary.GET(key, default_value)
```

3.4 Python translation

Resolved dictionary retrieval translates as:

```
dictionary.GET(key)
```

→

```
dictionary.get(key)
```

and:

```
dictionary.GET(key, default_value)
```

→

```
dictionary.get(key, default_value)
```

Ordinary user-defined member methods retain their method identifier and compile as ordinary Python method calls.

3.5 Backwards compatibility with v1.0

The v1.0 spellings:

```
dictionary~GET(key)
dictionary~GET(key, default_value)
```

remain accepted for backwards compatibility.

Under v1.1 compatibility normalisation they canonicalise to:

```
dictionary.GET(key)
dictionary.GET(key, default_value)
```

The `~GET` spelling is therefore accepted but non-canonical in v1.1.

4. Expression grammar update

The v1.1 primary/postfix expression model includes member calls in addition to indexed access, property access and legacy dictionary retrieval compatibility.

Conceptually:

```
postfix_expression ::=
primary_expression ,
{
"[" , expression , "]"
| "." , identifier
| "." , identifier , "(" , [ argument_list ] , ")"
| legacy_dictionary_get
} ;
```

`legacy_dictionary_get` represents the accepted-but-non-canonical v1.0 `~GET(...)` form.

5. Static semantic additions

1. NONE is a literal and requires no declaration.
2. Member calls are distinct from property access.
3. Dictionary GET is resolved from receiver type/evidence, not from method spelling alone.
4. A dictionary-resolved GET requires one or two arguments.
5. An ordinary method named GET remains legal on non-dictionary receivers.

6. Normalisation rules

Conservative v1.1 canonicalisation may perform:

```
None  -> NONE
none  -> NONE
```

when the token occurs outside a quoted string.

For an explicitly dictionary-backed receiver:

```
dictionary.get(key, default) -> dictionary.GET(key, default)
```

For legacy v1.0 syntax:

```
dictionary~GET(key, default) -> dictionary.GET(key, default)
```

These operations do not alter algorithmic behaviour.

Normalisation must not change quoted contents such as:

```
STRING("None")
```

and must not reinterpret an arbitrary non-dictionary `object.get(...)` as dictionary retrieval.

7. Version identification

This specification delta is frozen as:

```
Cambridge-Style Pseudocode — AJ_24038155 Dialect v1.1
```

Any later syntax addition, removal or semantic alteration requires a new dialect version.